Universidad Da Vinci de Guatemala
Facultad de Ingeniería, Industria y Tecnología
Aplicaciones Web

Evaluación Parcial II

Objetivo general

Evaluar la capacidad del estudiante para diseñar e implementar una aplicación web full stack aplicando arquitectura limpia, consumo de APIs, manejo de estado, persistencia de datos y buenas prácticas en Angular y Node.js.

Parte 1: Comprensión y Análisis (5 puntos)

Instrucciones: Responde de forma técnica y concisa.

1. Explica cómo se aplica Clean Architecture en un backend con Express:
   - ¿Cuál es la función de las capas domain, infrastructure e interfaces?

   R// Clean Architecture se aplica en un backend con Express mediante la separación del sistema en capas con responsabilidades bien definidas, evitando el acoplamiento entre la lógica del negocio y los detalles técnicos del framework o la base de datos. Su objetivo principal es mejorar la mantenibilidad, escalabilidad y claridad del código.

   Domain: contiene las entidades, reglas de negocio y contratos principales del sistema. Es la capa más importante, ya que representa la lógica central y no depende de tecnologías externas.

   Infrastructure: implementa los aspectos técnicos del sistema, como el acceso a base de datos, ORM, configuración y servicios externos. En este proyecto, Sequelize y MySQL pertenecen a esta capa.

   Interfaces: gestiona la interacción con el exterior, por ejemplo mediante controladores, rutas HTTP y manejo de solicitudes y respuestas en Express.
2. ¿Por qué es importante utilizar Reactive Forms y manejar las peticiones HTTP únicamente desde servicios en Angular?

   R// El uso de Reactive Forms es importante porque permite un control más estructurado del formulario, sus validaciones y su estado. Esto mismo facilita la escalabilidad, la reutilización de lógica y el mantenimiento del código, especialmente en aplicaciones con formularios más complejos.

   Por otra parte, manejar las peticiones HTTP únicamente desde servicios es una buena práctica porque entiendo que separa responsabilidades dentro de la aplicación. Los componentes se enfocan en la presentación y la interacción con el usuario, mientras que los servicios centralizan la comunicación con la API. Esto mejora significativamente la organización del proyecto, favorece la reutilización del código y facilita las pruebas y el mantenimiento como ya mencioné antes.

Parte 2: Aplicación de Conocimientos (90 puntos)

Descripción del proyecto

Desarrollar BiblioApp, una aplicación web para gestión y reserva de libros con:

- Backend: Node.js + Express + TypeScript
- ORM: Sequelize
- Base de datos: PostgreSQL
- Frontend: Angular 13 (o la versión que usted tenga instalada) + Angular Material
- Arquitectura: Clean Architecture

Restricciones obligatorias

Backend

- Uso de Express + TypeScript
- Implementación de Clean Architecture:
    - domain
    - infrastructure
    - interfaces

Nota: acá únicamente se mencionan las capas anteriores, sin embargo, usted debe implementar todas las que considere necesarias según lo visto en clase.

- Uso de Sequelize
- Configuración con dotenv
- CORS permitido solo para: http://localhost:4200

Frontend

- Angular Material
- Reactive Forms obligatorios → No se permitirán formularios basados en plantillas.
- HttpClient solo en servicios
- Uso de interfaces TypeScript (DTOs)
- Manejo con Observables y .subscribe()
- Puede usar módulos Standalone o No Standalone

Base de Datos (20 puntos)

Requisito:

Diseñar e implementar una base de datos en 3FN con las siguientes tablas:

- categorias
- libros
- autores
- libro_autores (relación N:M)
- usuarios
- reservas

Evaluación:

- Correcta normalización
- Relaciones bien definidas
- Integridad referencial

Nota: usted como estudiante deberá desarrollar la normalización que considere apropiada, siempre y cuando sea totalmente visible la normalización, recuerde que lo importante de la normalización son las relaciones dado que son las que se necesitarán mapear a nivel backend.

Backend – API REST (25 puntos)

Endpoints requeridos:

Categorías (CRUD)

- Crear
- Listar
- Actualizar
- Eliminar

Libros

- CRUD completo
- GET /libros/disponibles
  (libros sin reserva activa)

Reservas

- Listar reservas
- Crear reserva
- Cancelar reserva
- Devolver libro

Evaluación:

- Correcta implementación
- Uso adecuado de Sequelize
- Organización por capas (Clean Architecture)

Nota: no olvide que en este segmento deberá ser evidente el mapeo de las relaciones según el diseño de su normalización de BD.

Servicios e Interfaces Angular (10 puntos)

- Creación de servicios para:
  - libros
  - reservas
  - categorías

- Uso correcto de:
    - Interfaces (DTOs)
    - HttpClient en servicios
    - Observables

Frontend (35 puntos)

1. Login (incluido en catálogo) *(no evaluado como seguridad real)*

- Ruta: /login
- Fake authentication (sin JWT)
- Validación básica de formulario

Nota: Este apartado de login NO TIENE PUNTOS, dado que solo es por presentación del proyecto. El login puede ser fake o bien puede ser el que ya se desarrolló en clase.

2. Catálogo (20 puntos)

Ruta: /catalogo

Funcionalidades:

- Listar libros desde backend
- Mostrar solo disponibles (endpoint dedicado)
- Botón Reservar
- Actualización visual tras reservar

Evaluación:

- Consumo correcto de API
- UI funcional
- Manejo de estados

3. Mis Reservas (15 puntos)

Ruta: /mis-reservas

Funcionalidades:

- Listar reservas del usuario
- Botones:
    - Cancelar
    - Devolver

Evaluación:

- Correcta interacción con backend
- Actualización dinámica

Clean Architecture (10 puntos)

Evaluación de:

- Separación de responsabilidades
- Organización del proyecto
- Uso adecuado de capas

NOTA: aunque según el apartado de **Clean Architecture** indica que solo vale 10 puntos, si el proyecto backend no lo implementa correctamente, se aplicará deducción de puntos significativa, dado que esto es core para el desarrollo backend.

Parte 3: Reflexión y Pensamiento Crítico (5 puntos)

Responde de forma argumentada:

1. ¿Qué ventajas aporta usar Clean Architecture en comparación con una estructura tradicional?

   R// Clean Architecture mejora la separación de responsabilidades, evitando el acoplamiento entre lógica de negocio y tecnologías externas. Esto permite mayor mantenibilidad, escalabilidad y facilidad para realizar cambios (por ejemplo, cambiar base de datos o framework sin afectar el core).

   Además, facilita el testing al aislar la lógica del negocio y mejora la organización del código en proyectos de mayor tamaño.

2. ¿Qué limitaciones o problemas podrías encontrar al escalar esta aplicación a producción?

   R// Al escalar, pueden surgir problemas de concurrencia (reservas simultáneas), rendimiento (consultas sin optimización), y seguridad. También se deben considerar aspectos como escalabilidad del servidor, manejo de errores, logs y monitoreo, los cuales son esenciales en producción pero no siempre están cubiertos en implementaciones iniciales.

Link para repositorio: https://github.com/FernandoChavarriaUDV/BiblioApp-Parcial2.git

Entregable: el entregable deberá ser este mismo documento colocando en cada item su respectiva respuesta, por ejemplo, para el caso de todo lo que aplique que sea pregunta deberá colocar su respuesta, ejemplo:

1. **Explica cómo se aplica Clean Architecture en un backend con Express:**
    - **¿Cuál es la función de las capas domain, infrastructure e interfaces?: R// su respuesta**

Para el caso de los ítems prácticos, deberá grabar un video no mayor de 3 minutos mostrando el funcionamiento de la aplicación. Así mismo deberá invitar al usuario rarizandieta al repositorio de código, dicho repositorio debe tener ambos proyectos. De la misma forma, deberá colocar en el documento entregable el enlace que redirija al catedrático al repositorio de código.

NOTA: documentos que no cumplan con lo solicitado a nivel de entregable, se aplicará deducción de puntos significativa.

Rubrica de Evaluación

Criterio	Excelente	Bueno	Regular	Deficiente	Puntos
Comprensión conceptual	Respuestas claras, técnicas y precisas	Correctas con leves omisiones	Superficiales	Incorrectas	5
Base de datos (3FN)	Diseño correcto, normalizado y consistente	Pequeños errores	Relaciones incompletas	Incorrecto	20
API REST (Backend)	Endpoints completos, funcionales y bien estructurados	Funcional con mejoras	Incompleto	No funcional	25
Servicios e interfaces Angular	Uso correcto y consistente	Funcional con detalles	Uso parcial	Incorrecto	10
Catálogo (Frontend)	Funcional, dinámico y bien integrado	Funciona con detalles	Incompleto	No funcional	20
Mis Reservas	Flujo completo y funcional	Funciona parcialmente	Incompleto	No funciona	15
Clean Architecture	Implementación clara y correcta	Parcialmente aplicada	Poco clara	No aplicada	10
Pensamiento crítico	Reflexión profunda y bien argumentada	Buena	Superficial	Ausente	5